Bilkent University

Department of Electrical and Electronics Engineering

Lab 7 Report: Finite State Machine

Purpose

The purpose of this lab is designing a finite state machine (FSM) on a breadboard. I used some ICs, buttons and LEDs to make this.

Methodology

This FSM has 2 states, 2 inputs and 1 output. First, I make the design using a state transition diagram, state table and output table. After this, I build the circuit on the breadboard with logic gates, buttons and LEDs. Finally, I tested the behavior of the FSM by giving different inputs and checking the LED lights to see if it works.

Design Specifications

The FSM is basically a door lock system. The states are "locked" and "unlocked" which shows the current door status. The states are assigned like this:

- State UNLOCKED: "1"
- State LOCKED: "0"

I used two push buttons for the inputs (X1, X0). I didn't use LEDs to show the inputs in my circuit. Instead, I give the inputs manually by connecting jumper wires to 1 or 0 (VCC or Ground) on the breadboard. I marked these manual input connections on the figures clearly so you can see that nothing is missing in the circuit.

For showing the states, I put two LEDs. The top LED shows the UNLOCKED state and the bottom LED shows the LOCKED state. I marked their positions in the figures so it is easy to understand.

The door lock works depending on the current state:

LOCKED State: This state shows with the bottom LED. If the inputs are "00", "01", or "10", the door stays locked. Only the correct password input "11" changes the state to the UNLOCKED state. When the password is correct, the bottom LED turns off.

UNLOCKED State: This state shows with the top LED. In this state, the system has a memory feature. Inputs "00" and "11" keep the door open. To lock the door again, I used a 'touch-to-lock' logic. So, inputs "01" and "10" makes the system go back to LOCKED state. When you press a button to lock it, the top LED turns off.

Figure 1 shows the state transition diagram. Also, Figure 2 shows the state transition table and the output table.

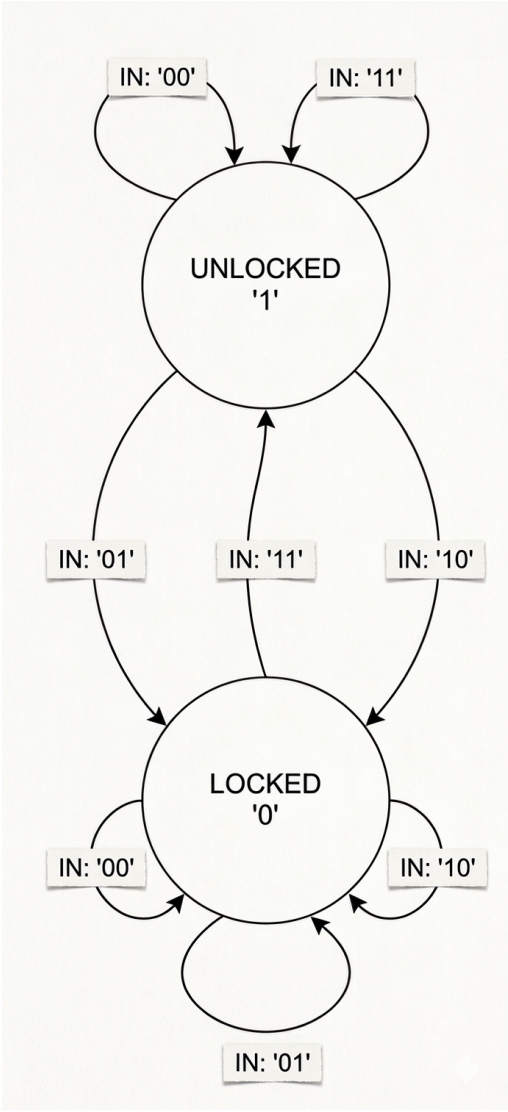


Figure 1: The State Transition Diagram

Q (Present State)	X1	X0	Q+	Output
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Figure 2: State Transition Table

When I look at the Karnaugh Map of the table, I found the logic for the D input as:

$$D = (X_1 \cdot X_0) + (Q \cdot \overline{X_1} \cdot \overline{X_0})$$

and the output is equal to Q+. This is a Moore Machine because the output only depends to the current states.

My design has 4 different logic gates: a NOT gate (SN74HC04N), a 2-input AND gate (SN74HC08N), a 2-input OR gate (SN74HC32N) and a Positive-Edge D Flip Flop (SN74HC74N). I connected the clock input of the D Flip Flop to a function generator. I set it to 1 Hz frequency so I can observe the changes with my eyes. The D Flip Flop stores the state, and the logic gates makes the combinational logic parts. The top LED is the UNLOCKED state (Q=1) and the bottom LED is the LOCKED state (Q=0).

Results

First, I build the FSM circuit on the breadboard using the components.

Then, I give different 2-bit inputs manually to test if the FSM works good. Figures 3 to 6 shows the inputs and the door states.

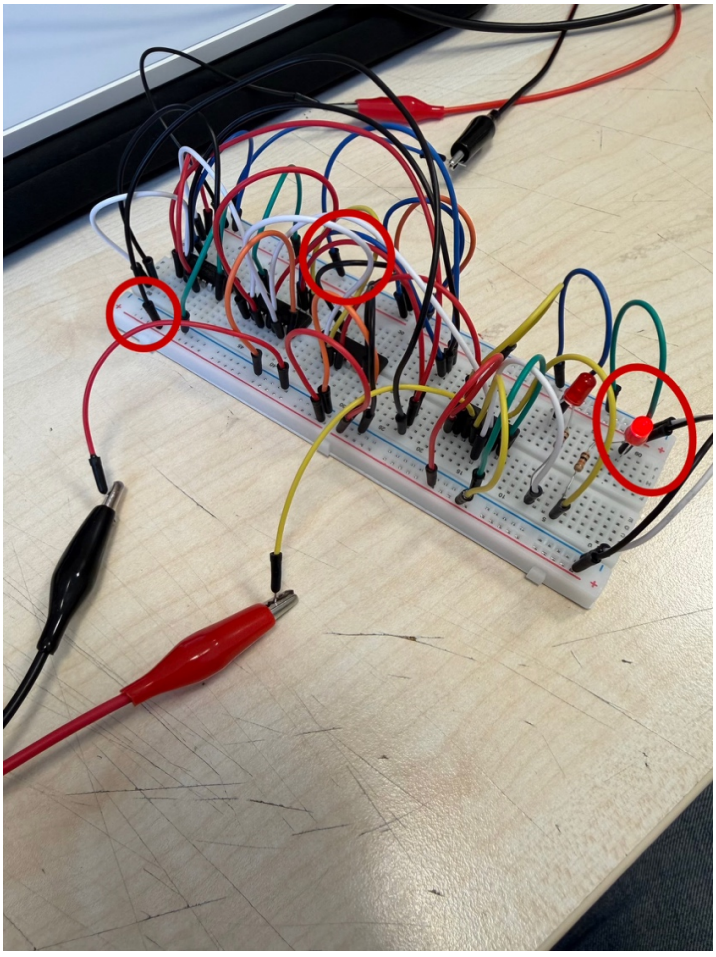


Figure 3: When Inputs x1x0: “10”, Q: “0” Output is 0 so the door is LOCKED

In this figure, the input is "10" (X1=1, X0=0). I put the left wire to the 1 line and the right wire to the 0 line. The door is still locked because the input is wrong. The system remains in the "Locked" state and only the bottom LED is on.

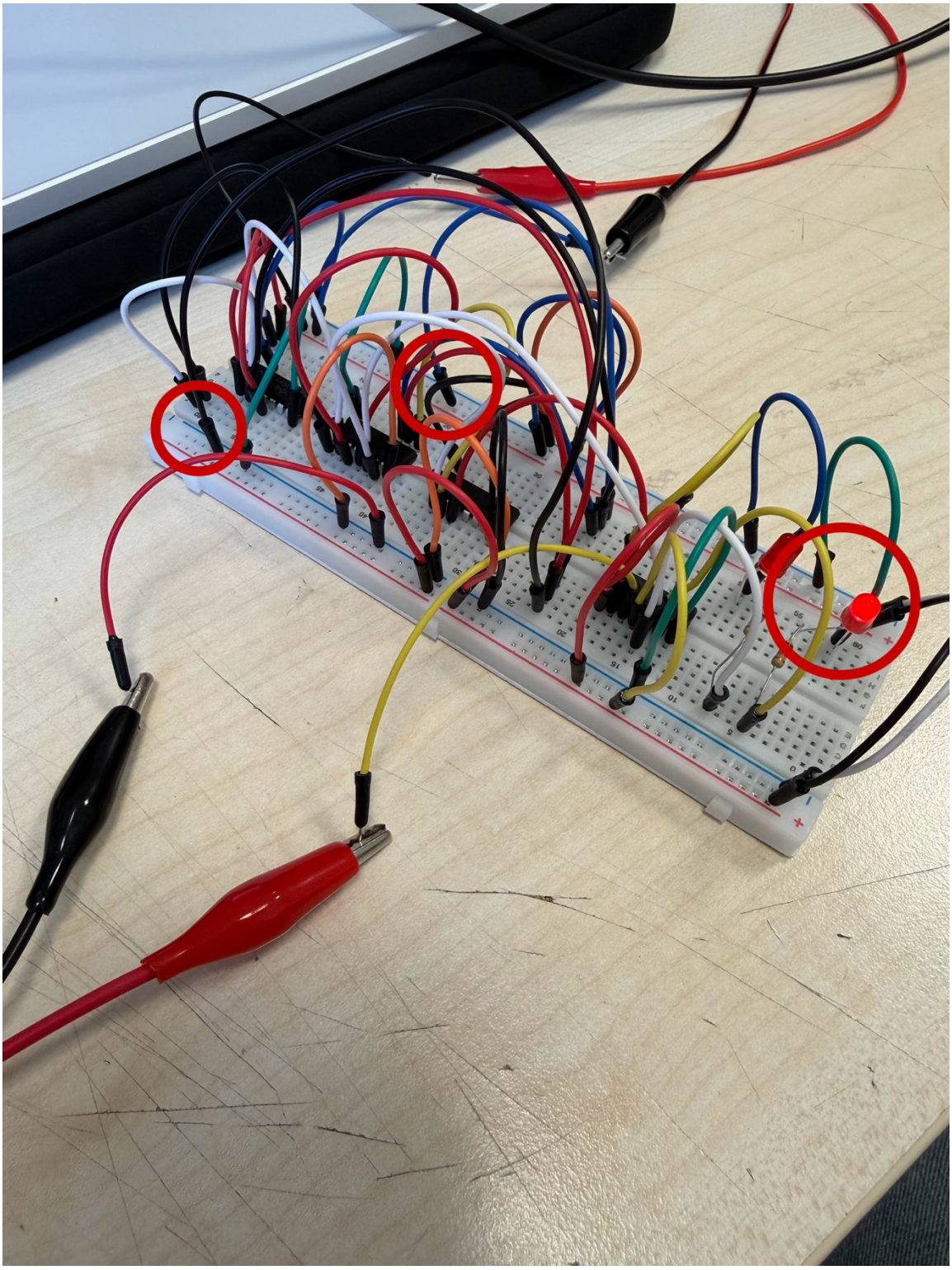


Figure 4: When Inputs x1x0: “01”, Q: “0” Output is 0 again so the LOCKED state remains

Here the input is "01" (X1=0, X0=1). I connected the left wire to the 0 line and the right wire to the 1 line manually. Because this is not the password, the system stays in the "Locked" state. As you can see, only the bottom LED is turning on.

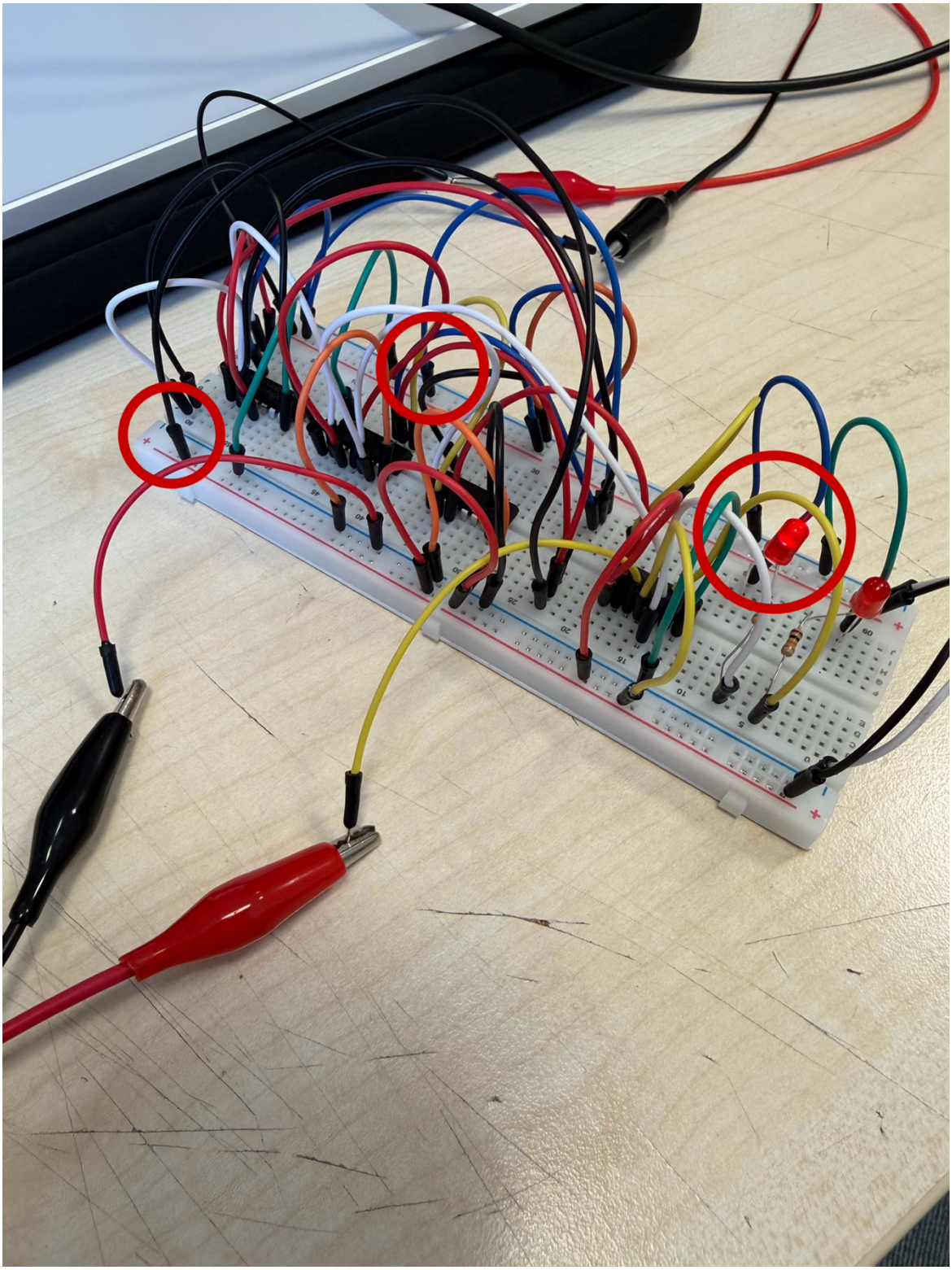


Figure 5: When Inputs x1x0: “11”, Q: “0” Output is 1 so the door UNLOCKED

Now I give the "11" input (X1=1, X0=1) by connecting both jumper wires to the 1 line. This is the correct password. The system takes this input and changes to the "Unlocked" state. The bottom LED turns off and the top LED turns on, which means the door is open now.

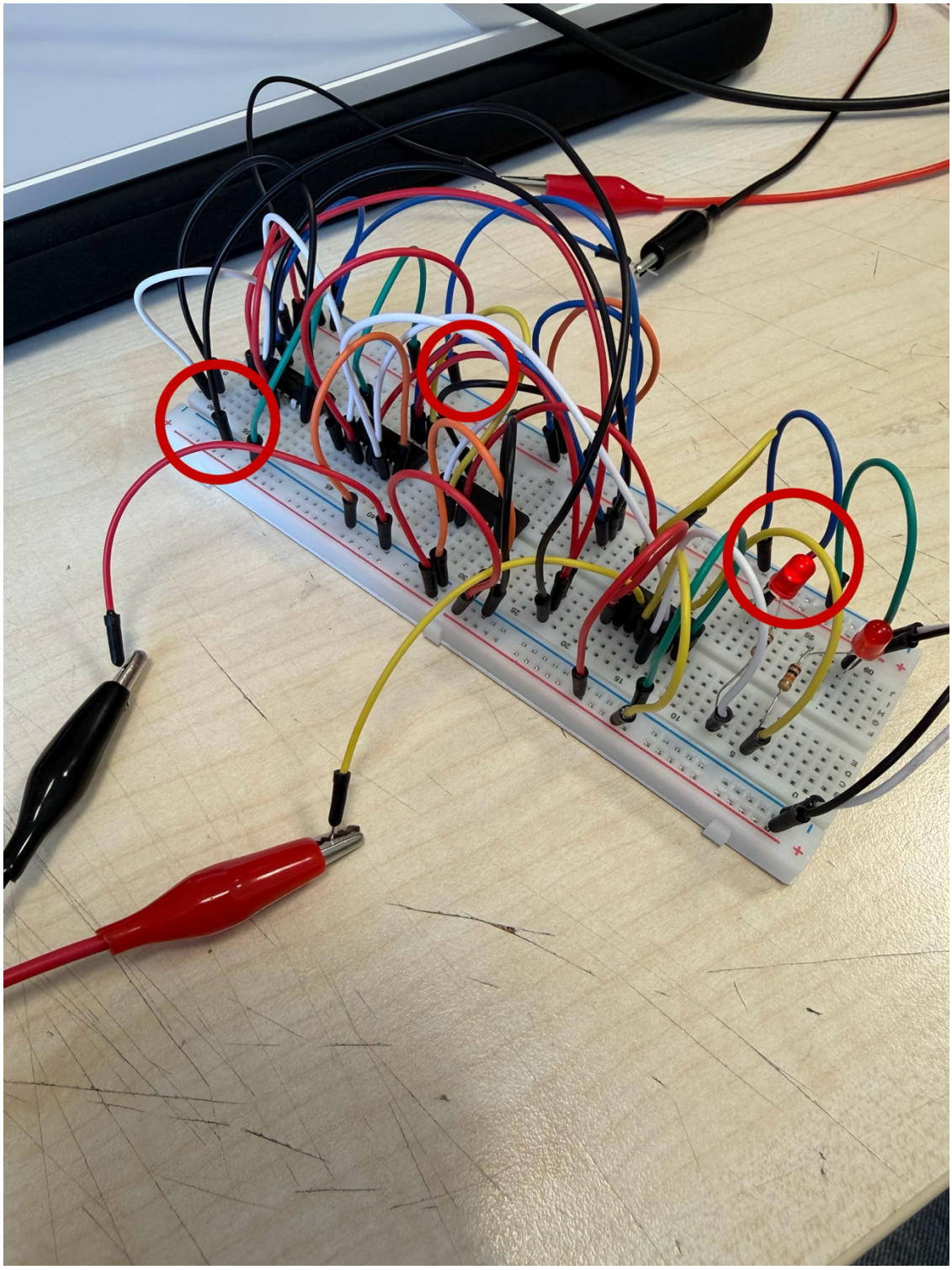


Figure 6: When Inputs x1x0: “00”, Q: “1” Output is 1 so the door state remains UNLOCKED

Finally, I changed the inputs back to "00" (X1=0, X0=0) after opening the door. Because our FSM design has a memory feature, the door doesn't close when I give "00". It stays in the "Unlocked" state. So, the top LED is still on and the bottom LED is off.

In general, the design worked correct and successfully.

Conclusion

In this experiment, I successfully designed and built a Moore-type Finite State Machine for a door lock system. First, I created the state transition tables and used Karnaugh maps to find the correct Boolean equations for the circuit. After that, I implemented this logic on the breadboard using a NOT gate, AND gate, OR gate, and a D Flip-Flop. As I mentioned before, I did not use input LEDs; instead, I used jumper wires manually to give 0 and 1 signals to the system.

The results of the experiment was very successful and consistent with my expected design. I observed that the FSM works exactly like the state transition diagram. When I gave the wrong inputs like "00", "01" or "10", the bottom LED stayed on and the door remained in the LOCKED state. But when I gave the correct "11" password with the wires, the top LED turned on and the door passed to the UNLOCKED state. Also, the memory feature worked perfectly because the door stayed open when I changed the inputs back to "00".

However, building the combinational logic was a little challenging because I had to use a lot of jumper wires, which made the breadboard look messy. Also, I observed an oscillation problem. If the "11" input is given continuously, the system state changes very fast and becomes unstable because "11" is also used as a password. Despite these minor physical challenges, the system behaved correctly. Overall, this lab was very helpful for me to understand how sequential circuits, state transitions, and memory elements work in real life on a breadboard.

References

- BTNcircuit.jpg
- LEDcircuit.jpg
- Bilkent University EEE Department, "EE 102 Digital Logic Design Lab Manual."
- Philips Semiconductors, "74HC74 Dual D-type flip-flop with set and reset," Datasheet. https://www.alldatasheet.com/datasheet-pdf/pdf/15661/PHILIPS/74HC74N.html
- Wikipedia, "Moore machine." https://en.wikipedia.org/wiki/Moore_machine
- Digilent, "Introduction to Finite State Machines." https://digilent.com/reference/learn/programmable-logic/tutorials/finite-state-machines/start
- Visual Paradigm, "What is State Machine Diagram?". https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-state-machine-diagram/